

Practice Test — Answer Review

English original of the 76-question mock exam — source: the “Practice Test” section of `guide_en.md`. Your selected answers, scored against the key, with every explanation shown.

68%

SCORE

52/76

CORRECT

52

RIGHT

24

WRONG

SCENARIO	SCORE	%
Multi-agent Research System	10/15	67%
Claude Code for Continuous Integration	7/15	47%
Code Generation with Claude Code	11/15	73%
Customer Support Agent	10/15	67%
Conversational AI Architecture Patterns	14/16	88%

A document analysis agent discovers that two credible sources contain directly contradictory statistics for a key metric: a government report states 40% growth, while an industry analysis states 12%. Both sources look credible, and the discrepancy could materially affect the research conclusions. How should the document analysis agent handle this situation most effectively?

Which approach is most effective?

- A Apply credibility heuristics to pick the most likely correct number, finish analysis with that value, and add a footnote mentioning the discrepancy.
- B Include both numbers in the analysis output without marking them as conflicting, letting the synthesis agent decide which to use based on broader context.
- C Stop analysis and immediately escalate to the coordinator, asking it to decide which source is more authoritative before continuing. **YOUR ANSWER**

- D Complete analysis with both numbers, explicitly annotate the conflict with source attribution, and let the coordinator decide how to reconcile the data before passing to synthesis. **CORRECT ANSWER**

Why D: This approach preserves separation of responsibilities: the analysis agent completes its core work without blocking, preserves both conflicting values with clear attribution, and correctly passes reconciliation to the coordinator, which has broader context.

The web-search and document-analysis agents have completed their tasks and returned results to the coordinator. What is the next step for creating an integrated research report?

Which next step is most appropriate?

- A Each agent sends its results directly to the report-writing agent, bypassing the coordinator.
- B The document analysis agent requests web-search results and merges them internally.
- C The coordinator passes both sets of results to the synthesis agent for a unified integration. **CORRECT ANSWER**
- D The coordinator concatenates the raw outputs from both agents and returns them as the final result.

Why C: In a coordinator-subagent architecture, the coordinator forwards both result sets to the synthesis agent for centralized integration, preserving control and ensuring high-quality merging.

A document analysis subagent frequently fails when processing PDF files: some have corrupted sections that trigger parsing exceptions, others are password-protected, and sometimes the parsing library hangs on large files. Currently, any exception immediately terminates the subagent and returns an error to the coordinator, which must decide whether to retry, skip, or fail the whole task. This causes excessive coordinator involvement in routine error handling. What architectural improvement is most effective?

Which improvement is most effective?

- A** Create a dedicated error-handling agent that monitors all failures via a shared queue and decides recovery actions, sending restart commands directly to subagents.
- B** Configure the subagent to always return partial results with a success status, embedding error details in metadata; the coordinator treats all responses as successful.
- C** Make the coordinator validate all documents before sending them to the subagent, rejecting documents that might cause failures.
- D** Implement local recovery in the subagent for transient failures and escalate to the coordinator only errors it cannot resolve, including attempted steps and partial results. **CORRECT ANSWER**

Why D: Handle errors at the lowest level capable of resolving them. Local recovery reduces coordinator workload while still escalating truly unrecoverable issues with full context and partial progress.

After running the system on "AI impact on creative industries," you observe that every subagent completes successfully: the web-search agent finds relevant articles, the document analysis agent summarizes them correctly, and the synthesis agent produces coherent text. However, final reports cover only visual art and completely miss music, literature, and film. In the coordinator logs, you see it decomposed the topic into three subtasks: "AI in digital art," "AI in graphic design," and "AI in photography." What is the most likely root cause?

What is the most likely root cause?

- A** The synthesis agent lacks instructions to detect coverage gaps.
- B** The document analysis agent filters out non-visual sources due to overly strict relevance criteria.
- C** The coordinator's task decomposition is too narrow, assigning subagents work that does not cover all relevant areas. **CORRECT ANSWER**
- D** The web-search agent's queries are insufficient and should be broadened to cover more sectors.

Why C: The coordinator decomposed a broad topic only into visual-art subtasks, missing music, literature, and film entirely. Since subagents executed their assignments correctly, the narrow decomposition is the obvious root cause.

The web-search subagent returns results for only 3 of 5 requested source categories (competitor sites and industry reports succeed, but news archives and social feeds time out). The document analysis subagent successfully processes all provided documents. The synthesis subagent must produce a summary from mixed-quality upstream inputs. Which error-propagation strategy is most effective?

Which error-propagation strategy is most effective?

- A** Continue synthesis using only successful sources and produce an output without mentioning which data was unavailable.
- B** The synthesis subagent returns an error to the coordinator, triggering a full retry or task failure due to incomplete data.
- C** The synthesis subagent asks the coordinator to retry timed-out sources with a longer timeout before starting synthesis.
- D** Structure the synthesis output with coverage annotations that indicate which conclusions are well-supported and where gaps exist due to unavailable sources. **CORRECT ANSWER**

Why D: Coverage annotations implement graceful degradation with transparency, preserving value from completed work while propagating uncertainty to enable informed decisions about confidence.

The document analysis subagent encounters a corrupted PDF file that it cannot parse. When designing the system's error handling, what is the most effective way to handle this failure?

Which approach is most effective?

- A** Return an error with context to the coordinator agent, allowing it to decide how to proceed. **CORRECT ANSWER**
- B** Silently skip the corrupted document and continue processing the remaining files to avoid interrupting the workflow.
- C** Automatically retry parsing the document three times with exponential backoff before reporting a failure.
- D** Throw an exception that terminates the entire research workflow.

Why A: Returning an error with context to the coordinator is the most effective approach because it lets the coordinator make an informed decision—skip the file, try an alternative parsing method, or notify the user—while maintaining visibility into the failure.

Production logs show a persistent pattern: requests like “analyze the uploaded quarterly report” are routed to the web-search agent 45% of the time instead of the document analysis agent. Reviewing tool definitions, you find that the web-search agent has a tool `analyze_content` described as “analyzes content and extracts key information,” while the document analysis agent has a tool `analyze_document` described as “analyzes documents and extracts key information.” How should you fix the misrouting problem?

How should you fix the misrouting problem?

- A** Add a pre-routing classifier that detects whether the user refers to uploaded files or web content before the coordinator decides on delegation.
- B** Rename the web-search tool to `extract_web_results` and update its description to “processes and returns information retrieved from web search and URLs.” **CORRECT ANSWER**
- C** Add few-shot examples to the coordinator prompt showing correct routing: “User uploads a quarterly report → document analysis agent” and “User asks about a web page → web-search agent.” **YOUR ANSWER**
- D** Expand the document analysis tool description with usage examples like “Use for uploaded PDFs, Word docs, and spreadsheets,” leaving the web-search tool unchanged.

Why B: Renaming the web-search tool to `extract_web_results` and updating its description to explicitly reference web search and URLs directly removes the root cause by eliminating semantic overlap between the two tool names and descriptions. This makes each tool’s purpose unambiguous, enabling the coordinator to reliably distinguish document analysis from web search.

A colleague proposes that the document analysis agent should send its results directly to the synthesis agent, bypassing the coordinator. What is the main advantage of keeping the coordinator as the central hub for all communication between subagents?

What is the main advantage of keeping the coordinator as the central hub?

- A** The coordinator can observe all interactions, handle errors uniformly, and decide what information each subagent should receive. **CORRECT ANSWER**
- B** The coordinator batches multiple requests to subagents, reducing total API calls and overall latency.
- C** Routing through the coordinator enables automatic retry logic that direct inter-agent calls cannot support.
- D** Subagents use isolated memory, and direct communication would require complex serialization that only the coordinator can perform.

Why A: The coordinator pattern provides centralized visibility into all interactions, uniform error handling across the system, and fine-grained control over what information each subagent receives—these are the primary advantages of a star-shaped communication topology.

The web-search subagent times out while researching a complex topic. You need to design how information about this failure is returned to the coordinator. Which error-propagation approach best enables intelligent recovery?

Which error-propagation approach best enables intelligent recovery?

- A** Return structured error context to the coordinator including the failure type, the query executed, any partial results, and potential alternative approaches. **CORRECT ANSWER**
- B** Catch the timeout within the subagent and return an empty result set marked as successful.
- C** Implement automatic exponential-backoff retries inside the subagent, only returning a generic “search unavailable” status after exhausting retries.
- D** Propagate the timeout exception directly to the top-level handler, terminating the entire research workflow.

Why A: Returning structured error context—including failure type, executed query, partial results, and alternative approaches—gives the coordinator everything needed to make intelligent recovery decisions (e.g., retry with a modified query or continue with partial results). It preserves maximum context for informed coordination-level decision-making.

In your system design, you gave the document analysis agent access to a general-purpose tool `fetch_url` so it could download documents by URL. Production logs show this agent now frequently downloads search engine results pages to perform ad hoc web search—behavior that should be routed through the web-search agent—causing inconsistent results. Which fix is most effective?

Which fix is most effective?

- A** Replace `fetch_url` with a `load_document` tool that validates that URLs point to document formats. **CORRECT ANSWER**
- B** Remove `fetch_url` from the document analysis agent and route all URL fetching through the coordinator to the web-search agent.
- C** Implement filtering that blocks `fetch_url` calls to known search engine domains while allowing other URLs.
- D** Add instructions to the document analysis agent prompt that `fetch_url` should only be used to download document URLs, not to search. **YOUR ANSWER**

Why A: Replacing a general-purpose tool with a document-specific tool that validates URLs against document formats fixes the root cause by constraining capability at the interface level. This follows the principle of least privilege, making undesired search behavior impossible rather than merely discouraged.

While researching a broad topic, you observe that the web-search agent and the document analysis agent investigate the same subtopics, leading to substantial duplication in their outputs. Token usage nearly doubles without a proportional increase in research breadth or depth. What is the most effective way to address this?

What is the most effective way to address this?

- A** Allow both agents to finish in parallel, then have the coordinator deduplicate overlapping results before passing them to the synthesis agent.
- B** The coordinator explicitly partitions the research space before delegating, assigning each agent distinct subtopics or source types. **CORRECT ANSWER**
- C** Implement a shared-state mechanism where agents log their current focus area so other agents can dynamically avoid duplication during execution.
- D** Switch to sequential execution where document analysis runs only after web search completes, using web-search results as context to avoid duplication. **YOUR ANSWER**

Why B: Having the coordinator explicitly partition the research space before delegating is most effective because it addresses the root cause—unclear task boundaries—before any work begins. It preserves parallelism while preventing duplicated effort and wasted tokens.

During research, the web-search subagent queries three source categories with different outcomes: academic databases return 15 relevant papers, industry reports return "0 results," and patent databases return "Connection timeout." When designing error propagation to the coordinator, which approach enables the best recovery decisions?

Which approach enables the best recovery decisions?

- A** Aggregate the results into a single success-percentage metric (e.g., "67% source coverage") with detailed logs available on demand.
- B** Report both "timeout" and "0 results" as failures requiring coordinator intervention.
- C** Retry transient failures internally and report only persistent errors.
- D** Distinguish access failures (timeout) that require a retry decision from valid empty results ("0 results") that represent successful queries. **CORRECT ANSWER**

Why D: A timeout (access failure) and "0 results" (valid empty result) are semantically different outcomes requiring different responses. Distinguishing them allows the coordinator to retry the patent database while accepting the industry reports "0 results" as a valid, informative finding.

Production monitoring shows inconsistent synthesis quality. When aggregated results are ~75K tokens, the synthesis agent reliably cites information from the first 15K tokens (web-search headlines/snippets) and the last 10K tokens (document analysis conclusions), but often misses critical findings in the middle 50K tokens—even when they directly answer the research question. How should you restructure the aggregated input?

How should you restructure the aggregated input?

- A** Summarize all subagent outputs to under 20K tokens before aggregation to keep content within the model's reliable processing range.
- B** Stream subagent results to the synthesis agent incrementally, processing web-search results first to completion, then adding document analysis results.
- C** Place a key-findings summary at the start of the aggregated input and organize detailed results with explicit section headings for easier navigation. **CORRECT ANSWER**
- D** Implement rotation that alternates which subagent's results appear first across research tasks to ensure both sources get equal top positioning over time. **YOUR ANSWER**

Why C: Putting a key-findings summary at the start leverages primacy effects so critical information sits in the most reliably processed position. Adding explicit section headings throughout helps the model navigate and attend to mid-input content, directly mitigating the "lost in the middle" phenomenon.

In testing, the combined output of the web-search agent (85K tokens including page content) and the document analysis agent (70K tokens including chains of thought) totals 155K tokens, but the synthesis agent performs best with inputs under 50K tokens. Which solution is most effective?

Which solution is most effective?

- A** Modify upstream agents to return structured data (key facts, quotes, relevance scores) instead of verbose content and reasoning. **CORRECT ANSWER**
- B** Add an intermediate summarization agent that condenses findings before passing them to synthesis.
- C** Have the synthesis agent process findings in sequential batches, maintaining state between calls.
- D** Store findings in a vector database and give the synthesis agent search tools to query during its work.

Why A: Modifying upstream agents to return structured data fixes the root cause by reducing token volume at the source while preserving essential information. It avoids passing bulky page content and reasoning traces that inflate tokens without improving the synthesis step.

In testing, you observe that the synthesis agent often needs to verify specific claims while merging results. Currently, when verification is needed, the synthesis agent returns control to the coordinator, which calls the web-search agent and then re-invokes synthesis with the results. This adds 2–3 extra loops per task and increases latency by 40%. Your assessment shows 85% of these verifications are simple fact checks (dates, names, stats) and 15% require deeper research. Which approach most effectively reduces overhead while preserving system reliability?

Which approach is most effective?

- A** Give the synthesis agent access to all web-search tools so it can handle any verification need directly without coordinator loops.
- B** Have the synthesis agent accumulate all verification needs and return them as a batch to the coordinator at the end, which then sends them all to the web-search agent at once.
- C** Have the web-search agent proactively cache extra context around each source during initial research in anticipation of synthesis needing verification.
- D** Give the synthesis agent a limited-scope `verify_fact` tool for simple checks, while routing complex verifications through the coordinator to the web-search agent. **CORRECT ANSWER**

Why D: A limited-scope fact-verification tool lets the synthesis agent handle 85% of simple checks directly, eliminating most loops, while preserving the coordinator delegation path for the 15% of complex verifications. This applies least privilege while significantly reducing latency.

Your CI pipeline runs the Claude Code CLI (in `--print` mode) using CLAUDE.md to provide project context for code review, and developers generally find the reviews substantive. However, they report that integrating findings into the workflow is difficult—Claude outputs narrative paragraphs that must be manually copied into PR comments. The team wants to automatically post each finding as a separate inline PR comment at the relevant place in code, which requires structured data with file path, line number, severity level, and suggested fix. Which approach is most effective?

Which approach is most effective?

- A** Add an "Output Format for Review" section to CLAUDE.md with examples of structured findings so Claude learns the expected format from project context.
- B** Use the CLI flags `--output-format json` and `--json-schema` to enforce structured findings, then parse the output to post inline comments via the GitHub API. **CORRECT ANSWER**
- C** Include explicit formatting instructions in the review prompt requiring each finding to follow a parseable template like `[FILE:path] [LINE:n] [SEVERITY:level] ...`. **YOUR ANSWER**
- D** Keep narrative review format but add a summarization step that uses Claude to generate a structured JSON summary of findings.

Why B: Using `--output-format json` with `--json-schema` enforces structured output at the CLI level, guaranteeing well-formed JSON with the required fields (file path, line number, severity, suggested fix) that can be reliably parsed and posted as inline PR comments via the GitHub API. It leverages built-in CLI capabilities designed specifically for structured output.

Your team uses Claude Code for generating code suggestions, but you notice a pattern: non-obvious issues—performance optimizations that break edge cases, cleanups that unexpectedly change behavior—are only caught when another team member reviews the PR. Claude’s reasoning during generation shows it considered these cases but concluded its approach was correct. Which approach directly addresses the root cause of this self-check limitation?

Which approach directly addresses the root cause?

- A** Run a second independent instance of Claude Code to review the changes without access to the generator’s reasoning. **CORRECT ANSWER**
- B** Enable extended thinking mode for the generation stage to allow more thorough deliberation before producing suggestions.
- C** Add explicit self-review instructions to the generation prompt asking Claude to critique its own suggestions before finalizing output.
- D** Include full test files and documentation in prompt context so Claude better understands expected behavior during generation. **YOUR ANSWER**

Why A: A second independent Claude Code instance without access to the generator’s reasoning directly addresses the root cause by avoiding confirmation bias. This “fresh eyes” perspective mirrors human peer review, where another reviewer catches issues the author rationalized.

Your code review component is iterative: Claude analyzes the changed file, then may request related files (imports, base classes, tests) via tool calls to understand context before providing final feedback. Your application defines a tool that lets Claude request file contents; Claude calls the tool, gets results, and continues analysis. You're evaluating batch processing to reduce API cost. What is the primary technical limitation when considering batch processing for this workflow?

What is the primary technical limitation?

A Batch processing does not include correlation IDs to map outputs back to input requests.

B The asynchronous model cannot execute tools mid-request and return results for Claude to continue analysis.

CORRECT ANSWER

C The Batch API does not support tool definitions in request parameters.

D The batch processing latency of up to 24 hours is too slow for pull request feedback, although the workflow would otherwise function.

YOUR ANSWER

Why B: A "fire-and-forget" asynchronous Batch API model has no mechanism to intercept a tool call during a request, execute the tool, and return results for Claude to continue analysis. This is fundamentally incompatible with iterative tool-calling workflows that require multiple tool request/response rounds within a single logical interaction.

Your CI/CD system runs three Claude-based analyses: (1) fast style checks on every PR that block merging until completion, (2) comprehensive weekly security audits of the entire codebase, and (3) nightly test-case generation for recently changed modules. The Message Batches API offers 50% savings but processing can take up to 24 hours. You want to optimize API cost while maintaining an acceptable developer experience. Which combination correctly matches each task to an API approach?

Which combination is correct?

- A** Use the Message Batches API for all three tasks to maximize 50% savings, configuring the pipeline to poll for batch completion.
- B** Use synchronous calls for PR style checks; use the Message Batches API for weekly security audits and nightly test generation. **CORRECT ANSWER**
- C** Use synchronous calls for all three tasks for consistent response times, relying on prompt caching to reduce costs across workloads.
- D** Use synchronous calls for PR style checks and nightly test generation; use the Message Batches API only for weekly security audits.

Why B: PR style checks block developers and require immediate responses via synchronous calls, while weekly security audits and nightly test generation are scheduled tasks with flexible deadlines that can tolerate up to a 24-hour batch window—capturing 50% savings for both.

Your automated reviews find real issues, but developers report the feedback is not actionable. Findings include phrases like “complex ticket routing logic” or “potential null pointer” without specifying what exactly to change. When you add detailed instructions like “always include concrete fix suggestions,” the model still produces inconsistent output—sometimes detailed, sometimes vague. Which prompting technique most reliably produces consistently actionable feedback?

Which prompting technique is most reliable?

- A** Further refine instructions with more explicit requirements for each part of the feedback format (location, issue, severity, proposed fix).
- B** Expand the context window to include more surrounding codebase so the model has enough information to propose concrete fixes.
- C** Implement a two-pass approach where one prompt identifies issues and a second generates fixes, allowing specialization. **YOUR ANSWER**
- D** Add 3–4 few-shot examples showing the exact required format: identified issue, location in code, concrete fix suggestion. **CORRECT ANSWER**

Why D: Few-shot examples are the most effective technique for achieving consistent output format when instructions alone produce variable results. Providing 3–4 examples that show the exact desired structure (issue, location, concrete fix) gives the model a concrete pattern to follow, which is more reliable than abstract instructions.

Your CI pipeline includes two Claude-based code review modes: a pre-merge-commit hook that blocks PR merge until completion, and a “deep analysis” that runs overnight, polls for batch completion, and posts detailed suggestions to the PR. You want to reduce API cost using the Message Batches API, which offers 50% savings but requires polling and can take up to 24 hours. Which mode should use batch processing?

Which mode should use batch processing?

- A** Only the pre-merge-commit hook.
- B** Only the deep analysis. **CORRECT ANSWER**
- C** Both modes.
- D** Neither mode.

Why B: Deep analysis is an ideal candidate for batch processing because it already runs overnight, tolerates delay, and uses a polling model before publishing results—matching the asynchronous, polling-based architecture of the Message Batches API while capturing 50% savings.

Your automated review analyzes comments and docstrings. The current prompt instructs Claude to “check that comments are accurate and up to date.” Findings often flag acceptable patterns (TODO markers, simple descriptions) while missing comments describing behavior the code no longer implements. What change addresses the root cause of this inconsistent analysis?

What change addresses the root cause?

- A** Include `git blame` data so Claude can identify comments that predate recent code changes.
- B** Add few-shot examples of misleading comments to help the model recognize similar patterns in the codebase.
- C** Filter TODO, FIXME, and descriptive comment patterns before analysis to reduce noise.
- D** Specify explicit criteria: flag comments only when the behavior they claim contradicts the code’s actual behavior. **CORRECT ANSWER**

Why D: Explicit criteria—flagging comments only when claimed behavior contradicts actual code behavior—directly addresses the root cause by replacing a vague instruction with a precise definition of what constitutes a problem. This reduces false positives on acceptable patterns and misses of truly misleading comments.

Your automated code review system shows inconsistent severity ratings—similar issues like null pointer risks are rated “critical” in some PRs but only “medium” in others. Developer surveys show growing distrust—many start dismissing findings without reading because “half are wrong.” High-false-positive categories erode trust in accurate categories. Which approach best restores developer trust while improving the system?

Which approach best restores developer trust?

- A** Temporarily disable high-false-positive categories (style, naming, documentation) and keep only high-precision categories while improving prompts. **CORRECT ANSWER**
- B** Keep all categories enabled but display confidence scores with each finding so developers can decide what to investigate.
- C** Keep all categories enabled and add few-shot examples to improve accuracy for each category over the next few weeks. **YOUR ANSWER**
- D** Apply a uniform strictness reduction across all categories to bring the overall false-positive rate down.

Why A: Temporarily disabling high-false-positive categories immediately stops trust erosion by removing noisy findings that cause developers to dismiss everything, while preserving value from high-precision categories like security and correctness. It also creates space to improve prompts for problematic categories before re-enabling them.

Your automated review generates test-case suggestions for each PR. Reviewing a PR that adds course completion tracking, Claude suggests 10 test cases, but developer feedback shows that 6 duplicate scenarios already covered by the existing test suite. What change most effectively reduces duplicate suggestions?

What change is most effective?

- A** Include the existing test file in context so Claude can determine what scenarios are already covered. **CORRECT ANSWER**
- B** Reduce the requested number of suggestions from 10 to 5, assuming Claude prioritizes the most valuable cases first.
- C** Add instructions directing Claude to focus exclusively on edge cases and error conditions rather than success paths.
- D** Implement post-processing that filters suggestions whose descriptions match existing test names via keyword overlap.

Why A: Including the existing test file fixes the root cause of duplication: Claude can only avoid suggesting already-covered scenarios if it knows what tests already exist. This gives Claude the information needed to propose genuinely new, valuable tests.

After an initial automated review identifies 12 findings, a developer pushes new commits to address issues. Re-running review produces 8 findings, but developers report that 5 duplicate previous comments on code that was already fixed in the new commits. What is the most effective way to eliminate this redundant feedback while maintaining thoroughness?

What is the most effective way to eliminate redundant feedback?

- A** Run review only when the PR is created and in the final pre-merge state, skipping intermediate commits. **YOUR ANSWER**
- B** Add a post-processing filter that removes findings that match previous ones by file paths and issue descriptions before posting comments.
- C** Restrict review scope to files changed in the most recent push, excluding files from earlier commits.
- D** Include previous review findings in context and instruct Claude to report only new or still-unresolved issues. **CORRECT ANSWER**

Why D: Including prior review findings in context lets Claude distinguish new problems from those already addressed in recent commits. This preserves review thoroughness while using Claude's reasoning to avoid redundant feedback on fixed code.

Your pipeline script runs `claude "Analyze this pull request for security issues"`, but the job hangs indefinitely. Logs show Claude Code is waiting for interactive input. What is the correct approach to run Claude Code in an automated pipeline?

What is the correct approach?

- A Add a `--batch` flag: `claude --batch "Analyze this pull request for security issues"`.
- B Add the `-p` flag: `claude -p "Analyze this pull request for security issues"`. CORRECT ANSWER**
- C Redirect stdin from `/dev/null`: `claude "Analyze this pull request for security issues" < /dev/null`.
- D Set the environment variable `CLAUDE_HEADLESS=true` before running the command.

Why B: The `-p` (or `--print`) flag is the documented way to run Claude Code non-interactively. It processes the prompt, prints the result to stdout, and exits without waiting for user input—ideal for CI/CD pipelines.

A pull request changes 14 files in an inventory tracking module. A single-pass review that analyzes all files together produces inconsistent results: detailed feedback on some files but shallow comments on others, missed obvious bugs, and contradictory feedback (a pattern is flagged in one file but identical code is approved in another file in the same PR). How should you restructure the review?

How should you restructure the review?

- A Run three independent full-PR review passes and flag only issues that appear in at least two of the three runs.
- B Split into focused passes: review each file individually for local issues, then run a separate integration-oriented pass to examine cross-file data flows. CORRECT ANSWER**
- C Require developers to split large PRs into smaller submissions of 3–4 files before running automated review.
- D Switch to a larger model with a bigger context window so it can pay sufficient attention to all 14 files in one pass.

Why B: Focused per-file passes address the root cause—attention dilution—by ensuring consistent depth and reliable local issue detection. A separate integration-oriented pass then covers cross-file concerns such as dependency and data-flow interactions.

Your automated code review averages 15 findings per pull request, and developers report a 40% false-positive rate. The bottleneck is investigation time: developers must click into each finding to read Claude's rationale before deciding whether to fix or dismiss it. Your CLAUDE.md already contains comprehensive rules for acceptable patterns, and stakeholders rejected any approach that filters findings before developers see them. What change best addresses investigation time?

What change best addresses investigation time?

- A** Require Claude to include its rationale and confidence estimate directly in each finding. **CORRECT ANSWER**
- B** Add a post-processor that analyzes finding patterns and automatically suppresses those that match historical false-positive signatures. **YOUR ANSWER**
- C** Categorize findings as "blocking issues" vs "suggestions," with different review requirements by level.
- D** Configure Claude to show only high-confidence findings, filtering uncertain flags before developers see them.

Why A: Including rationale and confidence directly in each finding reduces investigation time by letting developers quickly triage without opening each finding. It satisfies the "no filtering" constraint because all findings remain visible while accelerating developer decision-making.

Analysis of your automated code review shows large differences in false-positive rates by finding category: security/correctness findings have 8% false positives, performance findings 18%, style/naming findings 52%, and documentation findings 48%. Developer surveys show growing distrust—many start dismissing findings without reading because “half are wrong.” High-false-positive categories erode trust in accurate categories. Which approach best restores developer trust while improving the system?

Which approach best restores developer trust?

- A** Temporarily disable high-false-positive categories (style, naming, documentation) and keep only high-precision categories while improving prompts. **CORRECT ANSWER**
- B** Keep all categories enabled but display confidence scores with each finding so developers can decide what to investigate.
- C** Keep all categories enabled and add few-shot examples to improve accuracy for each category over the next few weeks. **YOUR ANSWER**
- D** Apply a uniform strictness reduction across all categories to bring the overall false-positive rate down.

Why A: Temporarily disabling high-false-positive categories immediately stops trust erosion by removing noisy findings that cause developers to dismiss everything, while preserving value from high-precision categories like security and correctness. It also creates space to improve prompts for problematic categories before re-enabling them.

Your team wants to reduce API costs for automated analysis. Currently, synchronous Claude calls support two workflows: (1) a blocking pre-merge check that must complete before developers can merge, and (2) a technical debt report generated overnight for review the next morning. Your manager proposes moving both to the Message Batches API to save 50%. How should you evaluate this proposal?

How should you evaluate this proposal?

- A** Move both to batch processing with fallback to synchronous calls if batches take too long.
- B** Move both workflows to batch processing with status polling to verify completion.
- C** Use batch processing only for technical debt reports; keep synchronous calls for pre-merge checks. **CORRECT ANSWER**
- D** Keep synchronous calls for both workflows to avoid issues with batch result ordering.

Why C: Message Batches API processing can take up to 24 hours with no latency SLA, which is acceptable for overnight technical debt reports but unacceptable for blocking pre-merge checks where developers wait. This matches each workflow to the right API based on latency requirements.

You asked Claude Code to implement a function that transforms API responses into an internal normalized format. After two iterations, the output structure still doesn't match expectations—some fields are nested differently and timestamps are formatted incorrectly. You described requirements in prose, but Claude interprets them differently each time.

Which approach is most effective for the next iteration?

- A** Write a JSON schema describing the expected output structure and validate Claude's output against it after each iteration. **YOUR ANSWER**
- B** Provide 2–3 concrete input-output examples showing the expected transformation for representative API responses. **CORRECT ANSWER**
- C** Rewrite requirements with more technical precision, specifying exact field mappings, nesting rules, and timestamp format strings.
- D** Ask Claude to explain its current understanding of the requirements to identify where interpretations diverge.

Why B: Concrete input-output examples remove ambiguity inherent in prose descriptions by showing Claude the exact expected transformation results. This directly addresses the root cause—misinterpretation of textual requirements—by providing unambiguous patterns for field nesting and timestamp formatting.

You need to add Slack as a new notification channel. The existing codebase has clear, established patterns for email, SMS, and push channels. However, Slack's API offers fundamentally different integration approaches—incoming webhooks (simple, one-way), bot tokens (support delivery confirmation and programmatic control), or Slack Apps (two-way events, requires workspace approval). Your task says “add Slack support” without specifying integration method or requiring advanced features like delivery tracking.

How should you approach this task?

- A** Start in direct execution mode using incoming webhooks to match the existing one-way notification pattern.
- B** Switch to planning mode to explore integration options and architectural implications, then present a recommendation before implementation. **CORRECT ANSWER**
- C** Start in direct execution mode by scaffolding a Slack channel class using existing patterns, deferring the integration method decision.
- D** Start in direct execution mode using a bot-token approach to ensure delivery confirmation is possible.

Why B: Slack integration has multiple valid approaches with significantly different architectural implications, and requirements are ambiguous. Planning mode lets you evaluate trade-offs among webhooks, bot tokens, and Slack Apps and align on an approach before implementation.

Your CLAUDE.md file has grown to 400+ lines containing coding standards, testing conventions, a detailed PR review checklist, deployment instructions, and database migration procedures. You want Claude to always follow coding standards and testing conventions, but apply PR review, deploy, and migration guidance only when doing those tasks.

Which restructuring approach is most effective?

- A Move all guidance into separate Skills files organized by workflow type, leaving only a brief project description in CLAUDE.md.
- B Keep everything in CLAUDE.md but use `@import` syntax to organize into separately maintained files by category.
- C Split CLAUDE.md into files under `.claude/rules/` with path-bound glob patterns so each rule loads only for the relevant file types. **YOUR ANSWER**

- D Keep universal standards in CLAUDE.md and create Skills for workflow-specific guidance (PR review, deploy, migrations) with trigger keywords.

CORRECT ANSWER

Why D: CLAUDE.md content loads in every session, ensuring coding standards and testing conventions always apply, while Skills are invoked on demand when Claude detects trigger keywords—ideal for workflow-specific guidance like PR review, deployment, and migrations.

You're tasked with restructuring your team's monolithic application into microservices. This impacts changes across dozens of files and requires decisions about service boundaries and module dependencies.

Which approach should you choose?

- A Switch to planning mode to explore the codebase, understand dependencies, and design the implementation approach before making changes. **CORRECT ANSWER**
- B Start in direct execution mode and switch to planning only after encountering unexpected complexity during implementation.
- C Start in direct execution mode and make incremental changes, letting implementation reveal natural service boundaries.
- D Use direct execution with detailed upfront instructions that specify each service structure.

Why A: Planning mode is the right strategy for complex architectural restructuring like splitting a monolith: it allows safe exploration and informed decisions about boundaries before committing to potentially expensive changes across many files.

Your team created a `/analyze-codebase` skill that performs deep code analysis—dependency scanning, test coverage counts, and code quality metrics. After running the command, team members report Claude becomes less responsive in the session and loses the context of the original task.

How do you most effectively fix this while keeping full analysis capabilities?

- A** Add `context: fork` in the skill frontmatter to run the analysis in an isolated subagent context. **CORRECT ANSWER**
- B** Add `model: haiku` in frontmatter to use a faster, cheaper model for analysis.
- C** Split the skill into three smaller skills, each producing less output.
- D** Add instructions to the skill to compress all results into a short summary before displaying them.

Why A: `context: fork` runs the analysis in an isolated subagent context so the large output does not pollute the main session's context window and Claude does not lose track of the original task. It preserves full analysis capability while keeping the main session responsive.

Your team uses a `/commit` skill in `.claude/skills/commit/SKILL.md`. A developer wants to customize it for their personal workflow (different commit message format, extra checks) without affecting teammates.

What do you recommend?

- A** Create a personal version under `~/.claude/skills/` with a different name, e.g., `/my-commit`. **YOUR ANSWER**
- B** Add conditional logic based on username in the project skill frontmatter.
- C** Create a personal version at `~/.claude/skills/commit/SKILL.md` with the same name. **CORRECT ANSWER**
- D** Set `override: true` in the personal skill frontmatter to prioritize it over the project version.

Why C: Personal skills take precedence over project skills with the same name. A personal skill at `~/.claude/skills/commit/SKILL.md` will override the team's project skill, allowing the developer to customize their workflow while maintaining the familiar `/commit` command name for their personal use. This approach is better than option A because it preserves the original command name, improving the developer's workflow without affecting teammates.

Your team has used Claude Code for months. Recently, three developers report Claude follows the guidance “always include comprehensive error handling,” but a fourth developer who just joined says Claude does not follow it. All four work in the same repo and have up-to-date code.

What is the most likely cause and fix?

- A** The guidance lives in the original developers’ user-level `~/.claude/CLAUDE.md` files, not in the project `.claude/CLAUDE.md`. Move the instruction to the project-level file **CORRECT ANSWER** so all team members receive it.
- B** The new developer’s `~/.claude/CLAUDE.md` contains conflicting instructions overriding project settings; they should delete the conflicting section.
- C** Claude Code learns per-user preferences over time; the new developer must repeat the requirement until Claude “remembers” it.
- D** Claude Code caches CLAUDE.md after first read; original developers use cached versions. Everyone should clear the Claude Code cache.

Why A: If the guidance was added only to the original developers’ user-level configs and not to the project-level `.claude/CLAUDE.md`, new team members won’t receive it. Moving it to the project-level configuration ensures all current and future team members automatically get the guidance.

You find that including 2–3 full endpoint implementation examples as context significantly improves consistency when generating new API endpoints. However, this context is useful only when creating new endpoints—not when debugging, reviewing code, or other work in the API directory.

Which configuration approach is most effective?

- A** Add endpoint examples and pattern documentation to the project CLAUDE.md so they are always available.
- B** Manually reference endpoint examples in every generation request by copying code into the prompt.
- C** Configure path-specific rules in `.claude/rules/api/` that include endpoint examples and activate when working in the API directory. **YOUR ANSWER**
- D** Create a skill that references the endpoint examples and contains pattern-following instructions, invoked on demand via a slash command. **CORRECT ANSWER**

Why D: A skill invoked on demand loads the example context only when generating new endpoints, not during unrelated tasks like debugging or review. This keeps the main context clean while preserving high-quality generation when needed.

Your team created a `/migration` skill that generates database migration files. It takes the migration name via `$ARGUMENTS`. In production you observe three issues: (1) developers often run the skill without arguments, causing poorly named files, (2) the skill sometimes uses database schema details from unrelated prior conversations, and (3) a developer accidentally ran destructive test cleanup when the skill had broad tool access.

Which configuration approach fixes all three problems?

- A** Use positional parameters `$1` and `$2` instead of `$ARGUMENTS` to enforce specific inputs, include explicit schema file references via `@` syntax for context control, and add a frontmatter description warning about destructive operations.
- B** Add `argument-hint` in frontmatter to request required parameters, use `context: fork` to isolate execution, and restrict `allowed-tools` to file-write operations. **CORRECT ANSWER**
- C** Split into `/migration-create` and `/migration-apply` skills, add validation instructions to request migration name if missing, and use different `allowed-tools` scopes for each.
- D** Add validation instructions in the skill `SKILL.md` to ensure `$ARGUMENTS` is a valid name, add prompts to ignore prior conversation context, and list prohibited operations to avoid.

Why B: This uses three separate configuration features to address each problem: `argument-hint` improves argument entry and reduces missing arguments, `context: fork` prevents context leakage from prior conversations, and `allowed-tools` constrains the skill to safe file-writing operations, preventing destructive actions.

Your codebase contains areas with different coding conventions: React components use functional style with hooks, API handlers use `async/await` with specific error handling, and database models follow the repository pattern. Test files are distributed across the codebase next to the code under test (e.g., `Button.test.tsx` next to `Button.tsx`), and you want all tests to follow the same conventions regardless of location.

What is the most supported way to ensure Claude automatically applies the correct conventions when generating code?

- A Put all conventions in the root `CLAUDE.md` under headings for each area and rely on Claude to infer which section applies.
- B Create skills in `.claude/skills/` for each code type, embedding conventions in each `SKILL.md`.
- C Place a separate `CLAUDE.md` file in each subdirectory containing conventions for that area.
- D Create rule files under `.claude/rules/` with YAML frontmatter specifying glob patterns to conditionally apply conventions based on file paths. **CORRECT ANSWER**

Why D: `.claude/rules/` files with YAML frontmatter and glob patterns (e.g., `**/*.test.tsx`, `src/api/**/*.ts`) enable deterministic, path-based convention application regardless of directory structure. This is the most supported approach for cross-cutting patterns like distributed test files.

You want to create a custom slash command `/review` that runs your team's standard code review checklist. It should be available to every developer when they clone or update the repository.

Where should you create the command file?

- A In `~/.claude/commands/` in each developer's home directory.
- B In the project repository under `.claude/commands/`. **CORRECT ANSWER**
- C In `.claude/config.json` as an array of commands.
- D In the root project `CLAUDE.md`.

Why B: Putting custom slash commands under `.claude/commands/` inside the project repository ensures they are version-controlled and automatically available to every developer who clones or updates the repo. This is the intended location for project-level custom commands in Claude Code.

Your team's CLAUDE.md grew beyond 500 lines mixing TypeScript conventions, testing guidance, API patterns, and deployment procedures. Developers find it hard to locate and update the right sections.

What approach does Claude Code support to organize project-level instructions into focused topical modules?

- A Define a `.claude/config.yaml` mapping file patterns to specific sections inside CLAUDE.md.
- B Create separate Markdown files in `.claude/rules/`, each covering one topic (e.g., `testing.md`, `api-conventions.md`).**
- C Split instructions into README.md files in relevant subdirectories that Claude automatically loads as instructions.
- D Create multiple files named CLAUDE.md at different levels of the directory tree, each overriding parent instructions.

CORRECT ANSWER

Why B: Claude Code supports a `.claude/rules/` directory where you can create separate Markdown files for topical guidance (e.g., `testing.md`, `api-conventions.md`), allowing teams to organize large instruction sets into focused, maintainable modules.

You create a custom skill `/explore-alternatives` that your team uses to brainstorm and evaluate implementation approaches before choosing one. Developers report that after running the skill, subsequent Claude responses are influenced by the alternatives discussion—sometimes referencing rejected approaches or retaining exploration context that interferes with actual implementation.

How should you most effectively configure this skill?

- A Use the `!` prefix in the skill to run exploration logic as a bash subprocess.
- B Add `context: fork` in the skill frontmatter.**
- C Split into two skills—`/explore-start` and `/explore-end`—to mark boundaries when exploration context should be discarded.
- D Create the skill in `~/.claude/skills/` instead of `.claude/skills/`.

CORRECT ANSWER

Why B: `context: fork` runs the skill in an isolated subagent context so exploration discussions do not pollute the main conversation history. This prevents rejected approaches and brainstorming context from influencing subsequent implementation work.

Your team wants to add a GitHub MCP server for searching PRs and checking CI status via Claude Code. Each of six developers has their own personal GitHub access token. You want consistent tooling across the team without committing credentials to version control.

Which configuration approach is most effective?

- A** Have each developer add the server in user scope via `claude mcp add --scope user`.
- B** Create an MCP server wrapper that reads tokens from a `.env` file and proxies GitHub API calls, then add the wrapper to the project `.mcp.json`.
- C** Add the server to the project `.mcp.json` using environment variable substitution (`${GITHUB_TOKEN}`) for auth and document the required environment variable in the project README. **CORRECT ANSWER**
- D** Configure the server in project scope with a placeholder token, then tell developers to override it in their local config.

Why C: A project `.mcp.json` with environment variable substitution is idiomatic: it provides a single version-controlled source of truth for MCP configuration while letting each developer supply credentials via environment variables. Documenting the variable makes onboarding easy without committing secrets.

You're adding error-handling wrappers around external API calls across a 120-file codebase. The work has three phases: (1) discover all call sites and patterns, (2) collaboratively design the error-handling approach, and (3) implement wrappers consistently. In Phase 1, Claude generates large output listing hundreds of call sites with context, quickly filling the context window before discovery finishes.

Which approach is most effective to complete the task while maintaining implementation consistency?

- A** Use an Explore subagent for Phase 1 to isolate verbose discovery output and return a summary, then continue Phases 2–3 in the main conversation. **CORRECT ANSWER**
- B** Do all phases in the main conversation, periodically using `/compact` to reduce context usage while moving through files.
- C** Switch to headless mode with `--continue`, passing explicit context summaries between batch calls to maintain continuity.
- D** Define the error-handling pattern in `CLAUDE.md`, then process files in batches across multiple sessions relying on the shared memory file for consistency.

Why A: An Explore subagent isolates the verbose discovery output in a separate context and returns only a concise summary to the main conversation. This preserves the main context window for the collaborative design and consistent implementation phases where retained context is most valuable.

While testing, you notice the agent often calls `get_customer` when users ask about order status, even though `lookup_order` would be more appropriate. What should you check first to address this problem?

What should you check first?

- A** Implement a preprocessing classifier to detect order-related requests and route them directly to `lookup_order`.
- B** Reduce the number of tools available to the agent to simplify choice.
- C** Add few-shot examples to the system prompt covering all possible order request patterns to improve tool selection.
- D** Check the tool descriptions to ensure they clearly differentiate each tool's purpose. **CORRECT ANSWER**

Why D: Tool descriptions are the primary input the model uses to decide which tool to call. When an agent consistently picks the wrong tool, the first diagnostic step is to verify that tool descriptions clearly separate each tool's purpose and usage boundaries.

Your agent handles single-issue requests with 94% accuracy (e.g., "I need a refund for order #1234"). But when customers include multiple issues in one message (e.g., "I need a refund for order #1234 and also want to update the shipping address for order #5678"), tool selection accuracy drops to 58%. The agent usually solves only one issue or mixes parameters across requests. What approach most effectively improves reliability for multi-issue requests?

What approach is most effective?

- A** Implement a preprocessing layer that uses a separate model call to decompose multi-issue messages into separate requests, handle each independently, and merge results. **YOUR ANSWER**
- B** Combine related tools into fewer universal tools.
- C** Add few-shot examples to the prompt demonstrating correct reasoning and tool sequencing for multi-issue requests. **CORRECT ANSWER**
- D** Implement response validation that detects incomplete answers and automatically reprompts the agent to resolve missed issues.

Why C: Few-shot examples that demonstrate correct reasoning and tool sequencing for multi-issue requests are most effective because the agent already performs well on single issues—what it needs is guidance on the pattern for decomposing and routing multiple issues and keeping parameters separated.

Production logs show that for simple requests like "refund for order #1234," your agent resolves the issue in 3–4 tool calls with 91% success. But for complex requests like "I was billed twice, my discount didn't apply, and I want to cancel," the agent averages 12+ tool calls with only 54% success—often investigating issues sequentially and fetching redundant customer data for each. What change most effectively improves handling of complex requests?

What change is most effective?

- A** Add explicit verification checkpoints between stages, requiring the agent to record progress after resolving each issue before moving to the next. **YOUR ANSWER**
- B** Reduce the number of tools by combining `get_customer`, `lookup_order`, and billing-related tools into a single `investigate_issue` tool.
- C** Decompose the request into separate issues, then investigate each in parallel using shared customer context before synthesizing a final resolution. **CORRECT ANSWER**
- D** Add few-shot examples to the system prompt demonstrating ideal tool-call sequences for various multi-faceted billing scenarios.

Why C: Decomposing into separate issues and investigating in parallel with shared customer context fixes both key problems: it eliminates redundant data retrieval by reusing shared context across issues and reduces total tool-call loops by parallelizing investigation before synthesizing a single resolution.

Your agent achieves 55% first-contact resolution, well below the 80% target. Logs show it escalates simple cases (standard replacements for damaged goods with photo proof) while trying to handle complex situations requiring policy exceptions autonomously. What is the most effective way to improve escalation calibration?

What is the most effective way to improve escalation calibration?

- A** Require the agent to self-rate confidence on a 1–10 scale before each response and automatically route to humans when confidence drops below a threshold.
- B** Deploy a separate classifier model trained on historical tickets to predict which requests need escalation before the main agent starts processing.
- C** Add explicit escalation criteria to the system prompt with few-shot examples showing when to escalate versus resolve autonomously. **CORRECT ANSWER**
- D** Implement sentiment analysis to determine customer frustration level and automatically escalate past a negative sentiment threshold.

Why C: Explicit escalation criteria with few-shot examples directly address the root cause—unclear decision boundaries between simple and complex cases. It's the most proportional, effective first intervention that teaches the agent when to escalate and when to resolve autonomously without extra infrastructure.

After calling `get_customer` and `lookup_order`, the agent has all available system data but still faces uncertainty. Which situation is the most justified trigger for calling `escalate_to_human`?

Which situation is most justified for escalation?

- A A customer wants to cancel an order shipped yesterday and arriving tomorrow. The agent should escalate because the customer might change their mind after receiving the package.
- B A customer claims they didn't receive an order, but tracking shows it was delivered and signed for at their address three days ago. The agent should escalate because presenting contradictory evidence could harm the customer relationship. **YOUR ANSWER**
- C A customer requests competitor price matching. Your policies allow price adjustments for price drops on your own site within 14 days, but say nothing about competitor prices. The agent should escalate for policy interpretation. **CORRECT ANSWER**
- D A customer message contains both a billing question and a product return. The agent should escalate so a human can coordinate both issues in one interaction.

Why C: This is a genuine policy gap: company rules cover price drops on your own site but do not address competitor price matching. The agent must not invent policy and should escalate for human judgment on how to interpret or extend existing rules.

Production logs show that in 12% of cases your agent skips `get_customer` and calls `lookup_order` directly using only the customer-provided name, sometimes leading to misidentified accounts and incorrect refunds. What change most effectively fixes this reliability problem?

What change is most effective?

- A Add few-shot examples showing that the agent always calls `get_customer` first, even when customers voluntarily provide order details.
- B Implement a routing classifier that analyzes each request and enables only a subset of tools appropriate for that request type.
- C Add a programmatic precondition that blocks `lookup_order` and `process_refund` until `get_customer` returns a verified customer identifier. **CORRECT ANSWER**
- D Strengthen the system prompt stating that customer verification via `get_customer` is mandatory before any order operations. **YOUR ANSWER**

Why C: A programmatic precondition provides a deterministic guarantee that required sequencing is followed. It's the most effective approach because it eliminates the possibility of skipping verification, regardless of LLM behavior.

Production metrics show that when resolving complex billing disputes or multi-order returns, customer satisfaction scores are 15% lower than for simple cases—even when the resolution is technically correct. Root-cause analysis shows the agent provides accurate solutions but inconsistently explains rationale: sometimes omitting relevant policy details, sometimes missing timeline info or next steps. The specific context gaps vary case by case. You want to improve solution quality without adding human oversight. What approach is most effective?

What approach is most effective?

- A** Add a self-critique stage where the agent evaluates a draft response for completeness—ensuring it resolves the customer’s issue, includes relevant context, and anticipates follow-up questions. **CORRECT ANSWER**
- B** Add a confirmation stage where the agent asks “Does this fully resolve your issue?” before closing, allowing customers to request additional information if needed.
- C** Upgrade the model from Haiku to Sonnet for complex cases, routing based on a defined complexity metric.
- D** Implement few-shot examples in the system prompt showing complete explanations for five common complex case types, demonstrating how to include policy context, timelines, and next steps. **YOUR ANSWER**

Why A: A self-critique stage (the evaluator-optimizer pattern) directly addresses inconsistent explanation completeness by forcing the agent to assess its own draft against concrete criteria—such as policy context, timelines, and next steps—before presenting it. This catches case-specific gaps without human oversight.

Production metrics show your agent averages 4+ API loops per resolution. Analysis reveals Claude often requests `get_customer` and `lookup_order` in separate sequential turns even when both are needed initially. What is the most effective way to reduce the number of loops?

What is the most effective way to reduce loops?

- A** Implement speculative execution that automatically calls likely-needed tools in parallel with any requested tool and returns all results regardless of what was requested.
- B** Increase `max_tokens` to give Claude more room to plan and naturally combine tool requests.
- C** Create composite tools like `get_customer_with_orders` that bundle common lookup combinations into single calls.
- D** Instruct Claude in the prompt to bundle tool requests into one turn and return all results together before the next API call. **CORRECT ANSWER**

Why D: Prompting Claude to bundle related tool requests into a single turn leverages its native ability to request multiple tools at once. It directly fixes the sequential-call pattern with minimal architectural change.

Production logs show a pattern: customers reference specific amounts (e.g., "the 15% discount I mentioned"), but the agent responds with incorrect values. Investigation shows these details were mentioned 20+ turns ago and condensed into vague summaries like "promotional pricing was discussed." What fix is most effective?

What fix is most effective?

- A** Increase the summarization threshold from 70% to 85% so conversations have more room before summarization triggers.
- B** Store full conversation history in external storage and implement retrieval when the agent detects references like "as I mentioned."
- C** Extract transactional facts (amounts, dates, order numbers) into a persistent "case facts" block included in every prompt outside the summarized history. **CORRECT ANSWER**
- D** Revise the summarization prompt to explicitly preserve all numbers, percentages, dates, and customer-stated expectations verbatim.

Why C: Summarization inherently loses precise details. Extracting transactional facts into a structured "case facts" block outside the summarized history preserves critical information so it's reliably available in every prompt regardless of how many turns have been summarized.

Your `get_customer` tool returns all matches when searching by name. Currently, when there are multiple results, Claude picks the customer with the most recent order, but production data shows this selects the wrong account 15% of the time for ambiguous matches. How should you address this?

How should you address this?

- A** Implement a confidence scoring system that acts autonomously above 85% confidence and requests clarification below the threshold.
- B** Instruct Claude to request an additional identifier (email, phone, or order number) when `get_customer` returns multiple matches before taking any customer-specific action. **CORRECT ANSWER**
- C** Modify `get_customer` to return only a single most-likely match based on a ranking algorithm, eliminating ambiguity.
- D** Add few-shot examples to the prompt demonstrating correct reasoning and tool sequencing for ambiguous matches.

Why B: Asking the user for an additional identifier is the most reliable way to resolve ambiguity because the user has definitive knowledge of their identity. One extra conversational turn is a small price to pay to eliminate a 15% error rate caused by choosing the wrong account.

Production logs show a consistent pattern: when customers include the word "account" in their message (e.g., "I want to check my account for an order I made yesterday"), the agent calls `get_customer` first 78% of the time. When customers phrase similar requests without "account" (e.g., "I want to check an order I made yesterday"), it calls `lookup_order` first 93% of the time. Tool descriptions are clear and unambiguous. What is the most likely root cause of this discrepancy?

What is the most likely root cause?

- A** The system prompt contains keyword-sensitive instructions that steer behavior based on terms like "account," creating unintended tool-selection patterns. **CORRECT ANSWER**
- B** The model's base training creates associations between "account" terminology and customer-related operations that override tool descriptions.
- C** The model needs more training data on multi-concept messages and should be fine-tuned on examples containing both account and order terminology.
- D** Tool descriptions need additional negative examples specifying when NOT to use each tool to prevent this keyword-induced confusion.

Why A: The systematic keyword-driven pattern (78% vs 93%) strongly indicates explicit routing logic in the system prompt reacting to the word "account" and steering the agent toward customer-related tools. Since tool descriptions are already clear, the discrepancy points to prompt-level instructions creating unintended behavioral steering.

Production logs show the agent often calls `get_customer` when users ask about orders (e.g., "check my order #12345") instead of calling `lookup_order`. Both tools have minimal descriptions ("Gets customer information" / "Gets order details") and accept similar-looking identifier formats. What is the most effective first step to improve tool selection reliability?

What is the most effective first step?

- A** Implement a routing layer that analyzes user input before each turn and preselects the correct tool based on detected keywords and ID patterns.
- B** Combine both tools into a single `lookup_entity` that accepts any identifier and internally decides which backend to query.
- C** Add few-shot examples to the system prompt demonstrating correct tool selection patterns, with 5–8 examples routing order-related queries to `lookup_order`.
- D** Expand each tool's description to include input formats, example queries, edge cases, and boundaries explaining when to use it versus similar tools. **CORRECT ANSWER**

Why D: Expanding tool descriptions with input formats, example queries, edge cases, and clear boundaries directly fixes the root cause—minimal descriptions that don't give the LLM enough information to distinguish similar tools. It's a low-effort, high-impact first step that improves the primary mechanism the LLM uses for tool selection.

You are implementing the agent loop for your support agent. After each Claude API call, you must decide whether to continue the loop (run requested tools and call Claude again) or stop (present the final answer to the customer). What determines this decision?

What determines this decision?

- A** Check the `stop_reason` field in Claude's response—continue if it is `tool_use` and stop if it is `end_turn`. **CORRECT ANSWER**
- B** Parse Claude's text for phrases like "I'm done" or "Can I help with anything else?"—natural language signals indicate task completion.
- C** Set a maximum iteration count (e.g., 10 calls) and stop when reached, regardless of whether Claude indicates more work is needed.
- D** Check whether the response contains assistant text content—if Claude generated explanatory text, the loop should terminate.

Why A: `stop_reason` is Claude's explicit structured signal for loop control: `tool_use` indicates Claude wants to run a tool and receive results back, while `end_turn` indicates Claude has completed its response and the loop should end.

Production logs show the agent misinterprets outputs from your MCP tools: Unix timestamps from `get_customer`, ISO 8601 dates from `lookup_order`, and numeric status codes (1=pending, 2=shipped). Some tools are third-party MCP servers you cannot modify. Which approach to data format normalization is most maintainable?

Which approach is most maintainable?

- A** Use a `PostToolUse` hook to intercept tool outputs and apply formatting transformations before the agent processes them. **CORRECT ANSWER**
- B** Modify tools you control to return human-readable formats and create wrappers for third-party tools.
- C** Create a `normalize_data` tool that the agent calls after every data retrieval to transform values.
- D** Add detailed format documentation to the system prompt explaining each tool's data conventions.

Why A: A `PostToolUse` hook provides a centralized, deterministic point to intercept and normalize all tool outputs—including third-party MCP server data—before the agent processes them. It's more maintainable because transformations live in code and apply uniformly, rather than relying on LLM interpretation.

Production logs show the agent sometimes chooses `get_customer` when `lookup_order` would be more appropriate, especially for ambiguous queries like "I need help with my recent purchase." You decide to add few-shot examples to the system prompt to improve tool selection. Which approach most effectively addresses the problem?

Which approach is most effective?

- A** Add explicit "use when" and "don't use when" guidance in each tool description covering ambiguous cases.
- B** Add examples grouped by tool—all `get_customer` scenarios together, then all `lookup_order` scenarios.
- C** Add 4–6 examples targeted at ambiguous scenarios, each with rationale for why one tool was chosen over plausible alternatives. **CORRECT ANSWER**
- D** Add 10–15 examples of clear, unambiguous requests demonstrating correct tool choice for typical scenarios for each tool.

Why C: Targeting few-shot examples at the specific ambiguous scenarios where errors occur, with explicit rationale for why one tool is preferable to alternatives, teaches the model the comparative decision process needed for edge cases. This is more effective than generic examples or declarative rules.

Your `remove_team_member` tool uses a `dry_run: boolean` parameter for previewing impacts before execution. Production monitoring shows the agent bypasses the preview step by calling with `dry_run=false` directly. You need to ensure every removal is preceded by a preview that the user explicitly confirms.

What is the most reliable approach?

- A** Add server-side validation that permits `dry_run=false` only when a `dry_run=true` call with identical parameters occurred within the past 60 seconds.
- B** Annotate the tool as requiring confirmation and configure the orchestration layer to prompt the user for approval before forwarding any calls to annotated tools.
- C** Add detailed instructions and few-shot examples to the tool description requiring the agent to always call with `dry_run=true` first and wait for user confirmation before calling again.
- D** Replace with two tools: `preview_remove_member` returns impact details and a single-use confirmation token; `execute_remove_member` requires that token, binding execution to the preview. **CORRECT ANSWER**

Why D: The two-tool token-binding approach makes it architecturally impossible to execute without a prior preview—the execute tool literally requires a token that only the preview tool can generate. This is the only approach that enforces the constraint at the code level rather than relying on LLM compliance with instructions (C), timing heuristics (A), or orchestration infrastructure (B).

Production monitoring shows your `search_catalog` tool fails 12% of the time: 8% are network timeouts that succeed when retried, and 4% are query syntax errors that never succeed regardless of retries. Currently both error types are returned identically, causing wasted retries.

How should you modify the tool's error handling?

- A** Add few-shot examples to your system prompt demonstrating how to distinguish network errors from syntax errors.
- B** Apply exponential backoff retry logic to all errors uniformly.
- C** Implement automatic retry with backoff for network timeouts inside the tool; return syntax errors immediately with parameter validation details. **CORRECT ANSWER**
- D** Return all errors with a `retryable` boolean flag and error type details.

Why C: Handling retries at the tool level for transient errors is the correct abstraction boundary—the tool has definitive knowledge of the error type and can implement deterministic retry logic without relying on the agent to interpret a flag (D) or follow prompt-level instructions (A). Uniform backoff (B) wastes time on syntax errors that will never succeed.

Over several turns discussing investment strategy, a user stated "I have a very low risk tolerance" and later "I want to maximize my returns." They now ask: "What should I invest in?"

Which approach best ensures the recommendation aligns with the user's actual priority?

- A** Surface the contradiction and ask the user to clarify which matters more. **CORRECT ANSWER**
- B** Provide separate recommendations for both scenarios.
- C** Proceed with the most recently stated preference.
- D** Recommend a balanced portfolio without addressing the conflict.

Why A: When user preferences directly contradict each other, surfacing the conflict and asking for clarification is the only way to guarantee the recommendation aligns with the user's true intent. Any other approach involves making an assumption that may be wrong—maximizing returns and low risk tolerance are fundamentally incompatible goals that require a human decision.

Users refine playlist preferences over multiple conversation turns. Two messages after a user said "I love jazz," Claude asks "What genres do you enjoy?"

What is the most likely cause?

- A Claude requires a vector database connection to maintain conversation memory.
- B The model's context window has been exceeded.
- C The Claude API requires a `session_id` parameter.
- D Your application isn't including prior messages in the `messages` array. **CORRECT ANSWER**

Why D: Claude has no server-side memory—every API call is stateless. Without including the full conversation history in the `messages` array of each request, Claude has no knowledge of prior turns. Vector databases (A) and `session_id` (C) are not part of Claude's architecture; context window overflow (B) is impossible for two-message exchanges.

After a 40-minute cooking session, the conversation reaches 78,000 tokens. History includes allergies, recipe scaling, clarified cooking terms, and general discussion. You must reduce tokens while preserving important information.

What approach best balances preservation with token reduction?

- A Summarize the entire conversation history.
- B Keep only the most recent 20,000 tokens.
- C Extract critical structured data (allergies, quantities, preferences), summarize general discussion, and keep recent exchanges verbatim. **CORRECT ANSWER**
- D Store the full conversation externally and retrieve relevant parts via semantic search.

Why C: The hybrid approach preserves the highest-value information at the lowest cost. Critical facts like allergies and recipe quantities are extracted into a compact structured block (preventing the precision loss that occurs during summarization), general discussion is summarized, and recent exchanges are kept verbatim for conversational coherence. Options A and B risk losing critical dietary information; D is architectural overkill for a single cooking session.

Users report that during extended conversations the assistant loses track of earlier topics and preferences. Your current implementation keeps only the last 25 message pairs.

What is the most effective solution?

A Hybrid approach: summarize older messages while keeping recent ones verbatim.

CORRECT ANSWER

B Vector similarity search over the full conversation history.

C Increase the window to 50 message pairs.

D Summarize dropped messages every turn and prepend the running summary.

Why A: The hybrid approach addresses both dimensions of the problem: retaining exact recent context (critical for conversational coherence) while maintaining a compressed representation of earlier preferences (preventing total loss when pairs are dropped). Increasing the window (C) simply delays the same problem. Vector search (B) may miss important context that isn't semantically similar to the current query. Full per-turn summarization (D) adds overhead and accumulates summarization errors.

Users report that latency increases and costs rise when conversations exceed 50 turns.

What is the primary cause?

A The entire conversation history is included with each API request.

CORRECT ANSWER

B The model generates progressively longer responses.

C Database operations slow down as history grows.

D The model builds an internal user profile requiring more processing.

Why A: Claude's API is fully stateless—every request must include the complete conversation history in the `messages` array. As conversations grow, each request carries more tokens, which directly increases both processing latency and cost. The model does not maintain any internal state between calls (D is false), and response length is not inherently tied to conversation length (B).

After three months of weekly sessions, conversation history grows to 85,000 tokens. When a user asks "What did we conclude about the theme of isolation?", the assistant gives generic answers instead of referencing previous discussions.

What is the most effective approach?

- A Rolling window truncation.
- B Progressive summarization capturing key conclusions.
- C Semantic embeddings with retrieval of relevant exchanges.
- D Add structured XML tags marking discussion conclusions.

YOUR ANSWER

CORRECT ANSWER

Why C: Semantic search over conversation history is the only approach that scales to three months of discussion while being able to surface specific relevant exchanges on demand. Rolling window (A) would discard most of the history. Progressive summarization (B) compresses discussions into abstractions that lose the specific conclusions users are asking about. XML tags (D) require restructuring all past content and don't solve the retrieval problem at this scale.

During QA testing, Claude follows system prompt guidelines for the first 10–15 turns, but later responses deviate. The conversation is still within token limits.

What is the best solution?

- A Move behavioral guidelines into the first user message.
- B Start a new conversation after 20 turns.
- C Insert user-role messages reinforcing guidelines at conversation breakpoints.
- D Use post-response validation to regenerate non-compliant responses.

CORRECT ANSWER

Why C: Periodic injection of behavioral reminders directly combats instruction drift by re-establishing constraints at regular intervals as conversation history accumulates. Moving guidelines to the first user message (A) reduces their authority. Starting a new conversation (B) destroys context. Post-response validation (D) is corrective rather than preventive and adds significant latency.

Your AI tutor has a 2,800-token system prompt defining teaching methodology and adaptation rules. After 12 turns, the assistant starts ignoring proficiency levels.

What is the most effective fix?

- ☐ A Inject reminders every 4–5 turns.
- ☒ B Replace verbose rules with few-shot examples demonstrating proficiency-level adaptation. **CORRECT ANSWER**
- ☐ C Place critical rules at the end of the system prompt.
- ☐ D Evaluate responses and regenerate if difficulty level mismatches.

Why B: A 2,800-token system prompt with declarative rules is vulnerable to drift because abstract rules require the model to reason about them on every turn. Replacing verbose rules with concrete few-shot examples that demonstrate correct proficiency-level adaptation gives the model clear behavioral patterns to match—this is more reliably followed across many turns than abstract instructions. Reminder injection (A) helps but addresses symptoms; end-placement (C) helps initially but not with turn-level drift; regeneration (D) is expensive and corrective.

Your assistant must maintain an enthusiastic tone, explain its reasoning, and ask clarifying questions. Where should these behavioral guidelines be defined?

Where should these behavioral guidelines be defined?

- ☐ A Prepended to each user message.
- ☒ B In the system prompt. **CORRECT ANSWER**
- ☐ C In the first assistant message.
- ☐ D In environment variables.

Why B: The system prompt is specifically designed for persistent behavioral constraints and guidelines that apply throughout the entire conversation. Prepending to each user message (A) is redundant overhead. The first assistant message (C) is unreliable because the model can deviate from its own prior statements. Environment variables (D) have no effect on model behavior.

Users report repetitive response openings like "Certainly!" and "I'd be happy to help!"

What is the most effective approach?

A Append a partial assistant message with a direct response opening.

CORRECT ANSWER

B Lower the temperature setting.

YOUR ANSWER

C Post-process responses to remove greetings.

D Add system prompt instructions to avoid those phrases.

Why A: Prefilling the assistant's response with the beginning of a direct answer prevents greeting patterns at the generation level—the model continues from the prefill rather than generating new opening phrases. System prompt instructions (D) can help but are less reliable since the model may still produce variants. Post-processing (C) is a fragile workaround. Temperature (B) controls randomness, not specific phrase patterns.

A webhook notifies your system that a user's package has shipped while the user is actively chatting. You want the assistant to incorporate this naturally into the next response.

What is the best approach?

A Add shipping status to the system prompt.

B Send an immediate synthetic user message.

C Force the assistant to call a status tool on each turn.

D Append the status update as a prefix to the next user message.

CORRECT ANSWER

Why D: Prefixing the status update to the next user message injects real-time context at a natural conversation boundary without disrupting the flow. Modifying the system prompt (A) requires rebuilding the session or is architecturally cumbersome. A synthetic user message (B) can break the natural dialogue flow and confuse attribution. Forcing a tool call each turn (C) is wasteful when events are rare.

Users frequently send requests like "Book a venue for the party." The assistant asks 4+ clarifying questions, causing 35% abandonment.

What approach best improves the trade-off?

- ☐ A Proceed with hidden defaults.
- ☐ B Ask all clarifying questions in one compound message.
- ☒ C State assumptions explicitly and proceed while inviting corrections.
- ☐ D Use a structured intake form.

CORRECT ANSWER

Why C: Stating assumptions explicitly and proceeding gives the user an immediate, useful response while preserving their ability to correct wrong assumptions. Hidden defaults (A) leave the user unaware of what was assumed. A compound question list (B) still demands upfront effort from the user. A structured form (D) adds more friction, not less—contradicting the goal of reducing abandonment.

Your assistant uses a contractor-persona system prompt. Early turns follow the rules, but by turn 7 the assistant gives generic advice. Conversation length is only 2,500 tokens.

What is the most likely cause?

- ☐ A System prompts only establish initial behavior.
- ☐ B Model attention weakens as turns accumulate.
- ☒ C Accumulated assistant responses dilute system prompt influence.
- ☐ D The system prompt is only sent once.

CORRECT ANSWER

Why C: As assistant responses accumulate in the conversation history, the proportion of text reflecting the system prompt's behavioral constraints decreases relative to the growing body of assistant-generated content. The model increasingly pattern-matches to its own prior outputs rather than the system prompt, compounding drift even at short token lengths. The system prompt is included in every API call (D is false as a standalone explanation), and model attention degradation (B) doesn't operate at 2,500 tokens.

Users ask vague requests like "Can you help with the report?" The assistant responds by asking multiple questions (which report? what help? deadline?), causing 40% abandonment.

What is the best solution?

A Make reasonable assumptions, state them explicitly, and offer to adjust.

CORRECT ANSWER

B Classify ambiguity with a smaller model before responding.

C Use predefined interpretations without stating assumptions.

D Limit the assistant to one clarifying question per turn.

Why A: Proceeding with reasonable stated assumptions eliminates the back-and-forth entirely while keeping the user informed and in control. Predefined silent interpretations (C) leave users confused when the response doesn't match their intent. A single-question limit (D) still requires turns of back-and-forth. A smaller classification model (B) adds latency and infrastructure complexity without solving the core UX problem.